

PROJECT MASTER DOCUMENT

Product, Business & Technical Overview

Document ID: PMD-001 **Version:** 1.0 **Status:** Baseline

Purpose of this document: Establish the product vision, business domains, technical direction, architecture principles, and delivery standards for the system. This document is written as professional project documentation and is suitable for inclusion in the project's Git repository.

1. Product Overview

The system is a modular, AI-enabled organizational platform that brings together project and process orchestration, administration, recognition and incentives, and holistic learning within a unified application ecosystem. The platform is designed around clearly separated business capabilities so that each capability can evolve independently while participating in defined end-to-end workflows.

2. Business Problem

Organizations often manage operational work, employee administration, recognition, and professional development through disconnected processes. This can make it difficult to connect day-to-day execution with employee growth, achievement, and organizational objectives.

The proposed platform addresses this gap by providing a unified set of business capabilities with explicit integration between operational activities, learning progress, and recognition. The architecture also emphasizes maintainability and knowledge continuity so that the system can be developed and maintained by teams of different sizes.

3. Product Objectives

- Provide a unified platform for project/process management, administration, recognition/incentives, and learning/development.
- Establish clear business-domain boundaries that support independent development and deployment.
- Enable meaningful integration between work completion, learning progress, and recognition.
- Provide AI-assisted capabilities where they create measurable business value.
- Maintain clear APIs, data ownership, security controls, testing, and operational documentation.
- Support a development model that remains maintainable when team size changes or developers join or leave.
- Keep the initial solution practical, understandable, and implementable without unnecessary infrastructure.

4. Core Business Modules

Module	Primary responsibility
Project & Process Orchestration	Manage projects, tasks, workflows, process states, dependencies and execution progress.
Administrative Hub	Manage users, roles, permissions and core administrative information.
Recognition & Incentives	Manage recognition, achievements, points, rewards and related business rules.
Holistic Learning Framework	Manage learning/development activities and progress across Physical, Social, Mental and Spiritual domains.

5. Module Summaries

5.1 Project & Process Orchestration

- Manage projects and work items.
- Support task assignment, dependencies and progress tracking.
- Represent configurable process/workflow states where required.

- Provide integration events or APIs for downstream business capabilities.

5.2 Administrative Hub

- Maintain user and organizational information.
- Provide role and permission management.
- Support authentication and authorization integration.
- Maintain administrative/audit information required by the system.

5.3 Recognition & Incentives

- Record recognition and achievements.
- Manage points or other incentive mechanisms where required.
- Manage reward definitions and redemption workflows.
- Ensure transactional operations are protected against duplicate processing.

5.4 Holistic Learning Framework

- Manage learning content, activities and user progress.
- Organize development activities under Physical, Social, Mental and Spiritual dimensions.
- Support competency/progress tracking.
- Provide AI-assisted recommendations or personalization where justified.

Professional boundary: the four dimensions are treated as configurable professional-development categories. The system is not intended to provide medical, psychological, or religious advice.

6. High-Level Architecture Direction

The solution is intended to use a modular microservice architecture. The final service decomposition will be confirmed after detailed requirements are completed; the four business modules are the starting domain boundaries rather than an assumption that every module must become exactly one service.

- Java and Spring Boot for backend services.
- REST APIs for synchronous business interactions where appropriate.
- Independent data ownership for services; uncontrolled shared database access is avoided.
- Authentication and authorization implemented with appropriate Spring Security mechanisms.
- An API Gateway may be introduced when it provides clear routing/security value.
- Asynchronous messaging may be introduced for workflows that benefit from event-driven communication.
- AI capabilities should be isolated behind a clear application boundary so the core business services are not tightly coupled to a single AI provider.

7. Technical Principles

- Use technology because it solves a requirement or provides meaningful engineering value.
- Prefer simple solutions for the first implementation; add infrastructure only when justified.
- Keep business logic separated from transport and persistence concerns.
- Use validation and consistent error handling at service boundaries.
- Document API contracts and important architectural decisions.
- Design for observability, testing and maintainability from the beginning.
- Keep service boundaries and contracts independent of individual developers.

8. AI Integration Principles

- AI must provide genuine business value rather than being included only as a chatbot/demo.
- Potential capabilities include personalized learning recommendations, progress analysis, summarization, classification, and contextual project assistance.
- AI output should be structured and validated before being used by business logic.
- Provider-specific code should be isolated where practical.
- AI failures, timeouts, unavailable models, privacy concerns and fallback behavior must be considered.
- AI must not be a mandatory paid dependency for the project.

9. Cost & Integration Constraint

Hard requirement: The project must be buildable, runnable, tested and demonstrated with a zero-budget approach.

- Prefer free and open-source software and locally runnable tools.
- Paid subscriptions or paid APIs must not be required for core functionality.
- Cloud free tiers may be considered only when they are genuinely sufficient and do not create a mandatory payment dependency.
- AI development must have a free/local path. External paid AI APIs must not be required.
- Every proposed external integration must identify its cost and provide a no-cost alternative or local fallback where practical.
- Architecture decisions must avoid unexpected recurring charges.

10. Team-Independent Architecture

The architecture belongs to the product, not to a particular team size. The system should support one developer, two developers, four developers, or a larger team without redesigning the business boundaries.

- Modules/services have explicit responsibilities and interfaces.
- API contracts and integration behavior are documented.
- Data ownership is documented.
- Each service has setup, testing and deployment instructions.
- Important decisions are recorded using Architecture Decision Records (ADRs).
- Developers can join later and understand a service from the repository.
- If a developer leaves, another developer can take over using the documented codebase, contracts and operating instructions.

11. Security Direction

- Authentication and authorization are required for protected business operations.
- Role/permission rules must be explicit.
- APIs must validate and sanitize inputs appropriately.
- Secrets must not be committed to Git.
- Sensitive configuration must be supplied through environment/configuration mechanisms.
- Audit requirements should be defined for security-sensitive and administrative actions.

12. Testing Direction

- Unit tests for business logic.
- Integration tests for persistence and important service behavior.
- API/controller tests for external contracts.

- Contract tests where service-to-service compatibility needs explicit protection.
- End-to-end tests for critical business workflows.
- AI integration tests that do not require paid external calls.

13. Documentation & Repository Standards

- Maintain a clear README for the project and each service.
- Document setup, configuration, APIs, database ownership, testing and deployment.
- Maintain architecture diagrams and API specifications.
- Use ADRs for significant architectural decisions.
- Keep SQL/schema and OpenAPI sources version-controlled where applicable.
- Documentation should be sufficient for another developer to continue the project without undocumented handover knowledge.

14. Development Lifecycle

1. Confirm detailed business requirements.
2. Define module use cases and business rules.
3. Finalize service boundaries.
4. Design database schemas and ownership.
5. Design API contracts.
6. Define service communication.
7. Define security.
8. Implement services incrementally.
9. Implement meaningful AI functionality.
10. Build automated tests.
11. Containerize and configure local execution.
12. Integrate services.
13. Validate end-to-end workflows.
14. Document final decisions and deployment.

15. Professional Documentation Status

This document is the high-level baseline. Detailed requirements, module specifications, database design, API specifications, AI design, security design, testing strategy and deployment documentation will be created as separate documents. Any feature not yet confirmed must be labeled as a proposal, assumption or open question rather than silently treated as a final requirement.

16. Next Document

Document 02 — Software Requirements Specification (SRS): define the exact functional requirements, actors, use cases, workflows, business rules, priorities, assumptions and acceptance criteria for all four modules. The database and API documents should be derived from the finalized SRS.